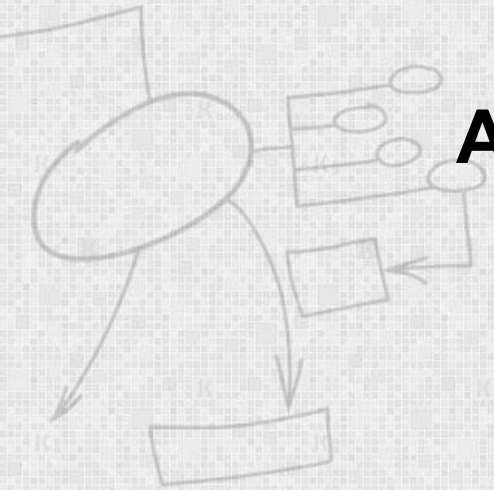
EC-360
SOFTWARE ENGINEERING

Lecture 2

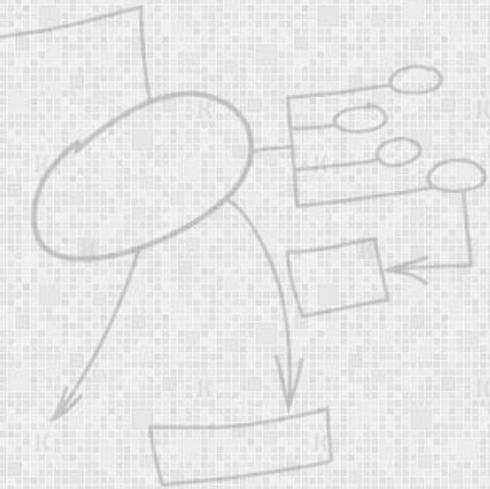
Agile Software Development

By
Dr Wasi Haider Butt



Today's Lecture: Agenda

- Agile Manifesto
- Agile Development
- Agile Development Models



The Manifesto for Agile Software Development

In 2001, Kent Beck and 16 other noted software developers, writers, and consultants (referred to as the "Agile Alliance") signed the "Manifesto for Agile Software Development." It stated:

"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- ***Individuals and interactions*** **over processes and tools**
- ***Working software*** **over comprehensive documentation**
- ***Customer collaboration*** **over contract negotiation**
- ***Responding to change*** **over following a plan**

That is, while there is value in the items on the right, we value the items on the left more."

Kent Beck et al

Introduction
Agile Software Development

Agile methods were developed in an effort to overcome perceived and actual weaknesses in conventional software engineering

Agile development can provide important benefits, but it is not applicable to all projects, all products, all people, and all situations.

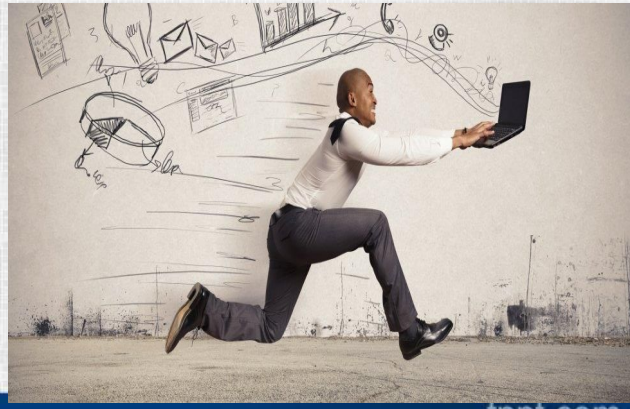
It is also *not* negating solid software engineering practice and can be applied as an over-riding philosophy for all software work.

What is “Agility”?

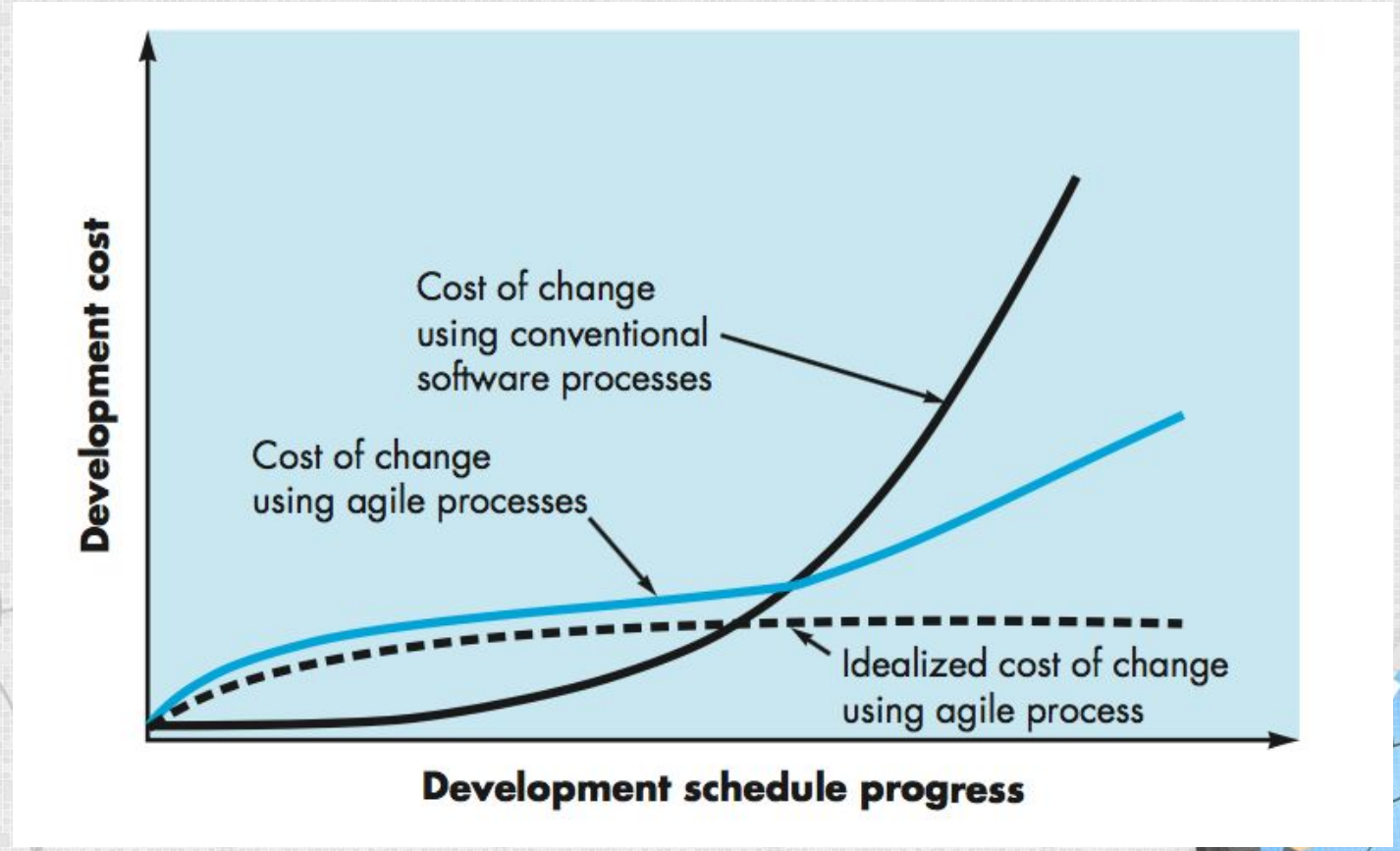
- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team

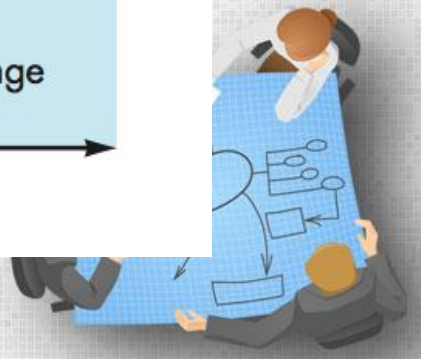
Yielding …

- Quick, incremental delivery of software



Agility and the Cost of Change



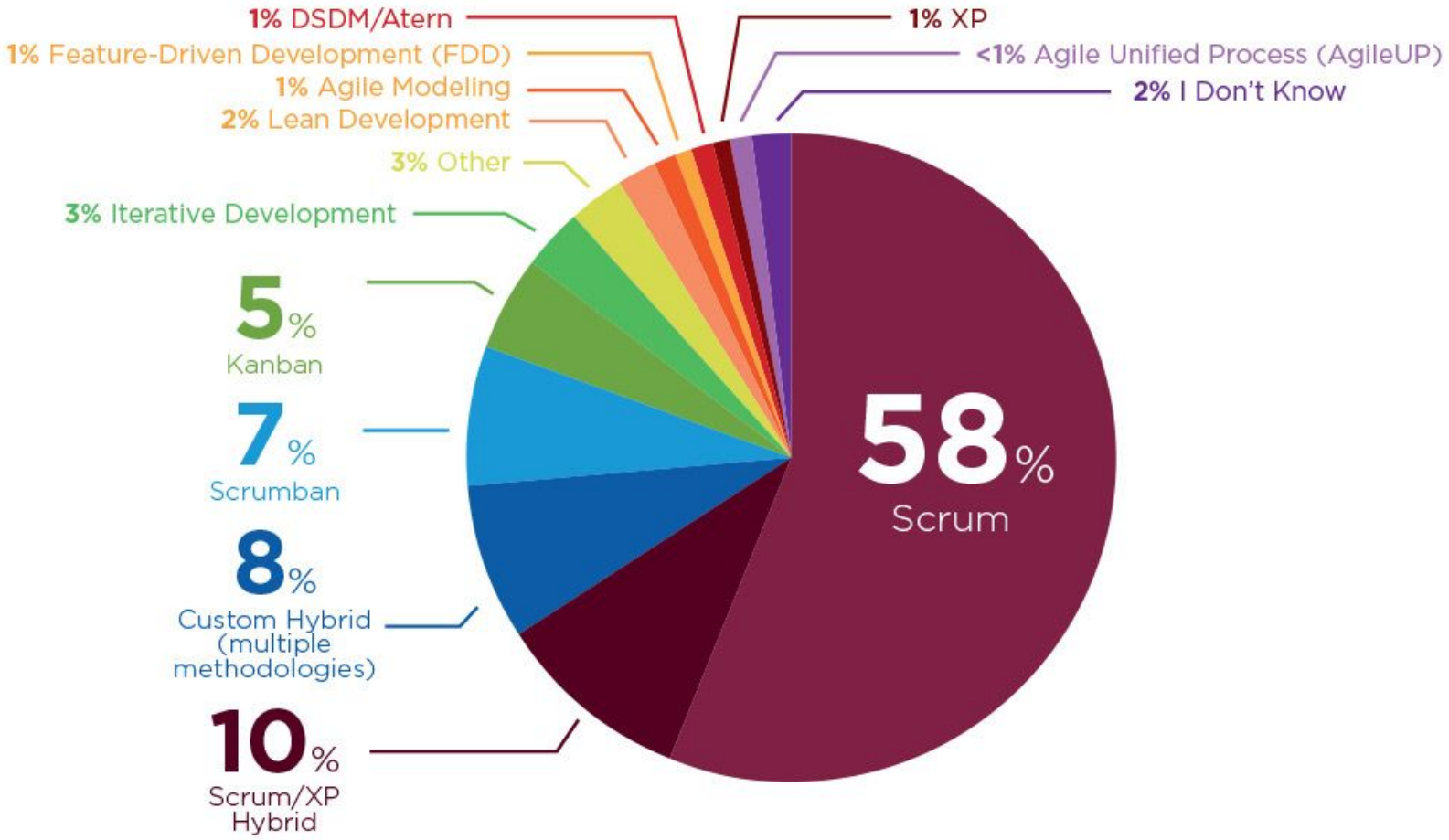


Extreme Programming (XP)

- The most widely used agile process (IN PAST), originally proposed by Kent Beck
- Extreme Programming uses an object-oriented approach as its preferred development paradigm
- Includes a set of rules and practices that occur within the context of four framework activities: planning, design, coding, and testing.

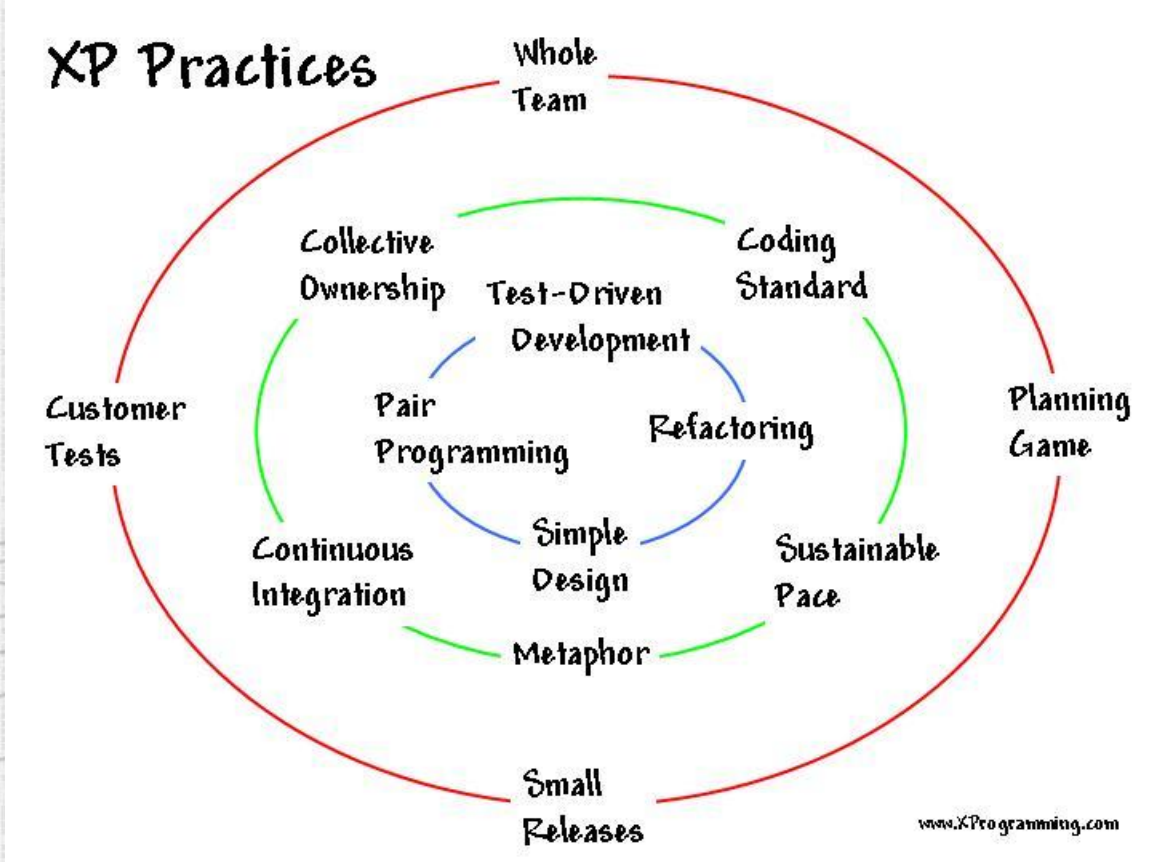
Agile Methodologies Used

When asked what agile methodology is followed most closely, nearly 70% of respondents practice Scrum (58%) or Scrum/XP hybrid (10%).



SOURCE: VERSIONONE 10TH ANNUAL STATE OF AGILE™ REPORT

XP Practices



(Source: http://www.xprogramming.com/xpmag/whatisxp.htm)

XP Practices: Whole Team

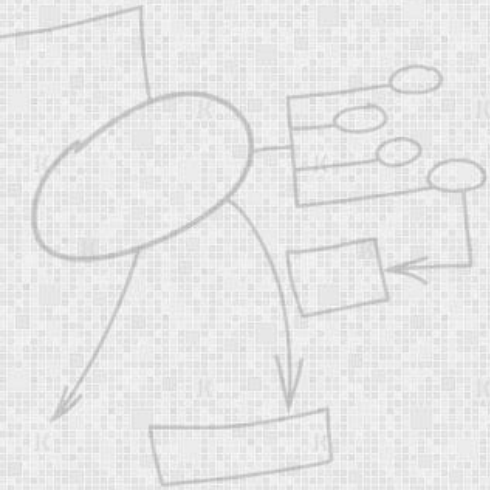
- All contributors to an XP project are one team
- Must include a business representative--the 'Customer'
  - Provides requirements
  - Sets priorities
  - Steers project
- Team members are programmers, testers, analysts, managers

XP Practices: Planning Game

- An agile development meeting
- Customer is involved in planning release
- XP Iteration/Release Planning
  - Customer presents required features
  - Programmers estimate difficulty
  - Two week iterations
  - Programmers break features down into tasks
  - Team members sign up for tasks
  - Running software at end of each iteration

XP Practices: Customer Tests

- The Customer defines one or more automated acceptance tests for a feature
- Team builds these tests to verify that a feature is implemented correctly

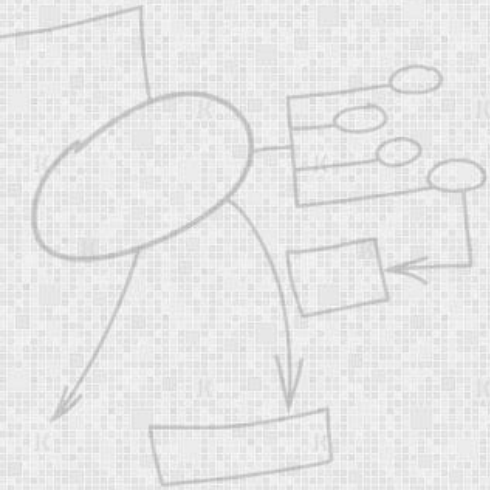


XP Practices: Small Releases

- Team releases running, tested software every iteration
- Releases are small and functional
- The Customer can evaluate or in turn, release to end users, and provide feedback
- Important thing is that the software is visible and given to the Customer at the end of every iteration

XP Practices: Simple Design

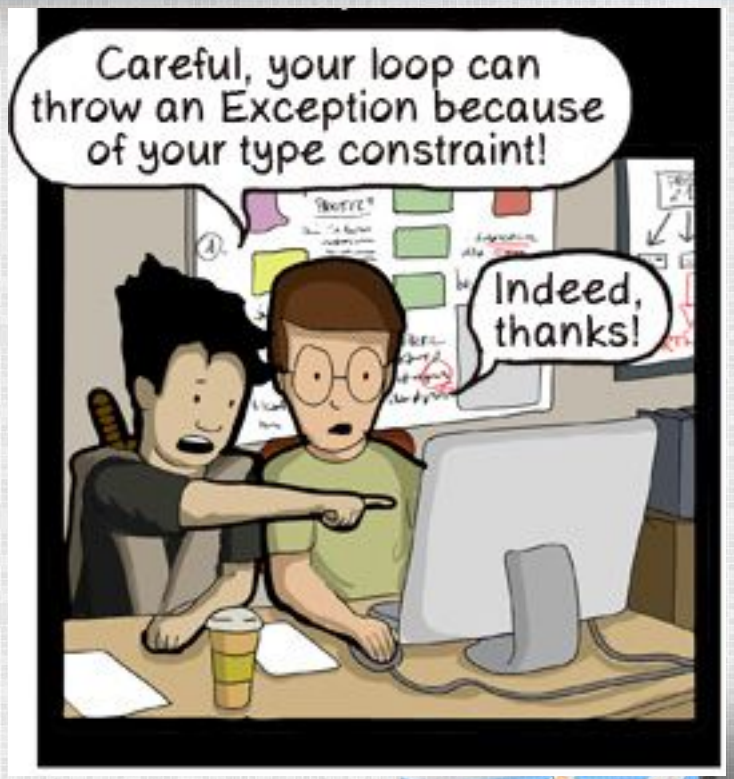
- Simple design
- Design suited to current functionality
- Not a one-time thing nor an up-front thing
- Teams design and revise design through refactoring, through the course of the project



XP Practices: Pair Programming

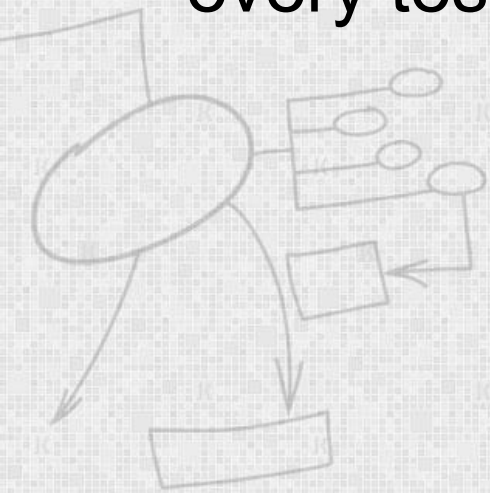
- All production software is built by two programmers, sitting side by side, at the same machine
- All production code is therefore reviewed by at least one other programmer
- Research into pair programming shows that pairing produces better code at the same time as programmers working singly
- Pairing also communicates knowledge throughout the team

XP Practices: Pair Programming



XP Practices: Test-Driven Development

- Teams practice TDD by working in short cycles of adding a test, and then making it work
- Easy to produce code with 100 percent test coverage
- Each time a pair releases code to the repository, every test must run correctly

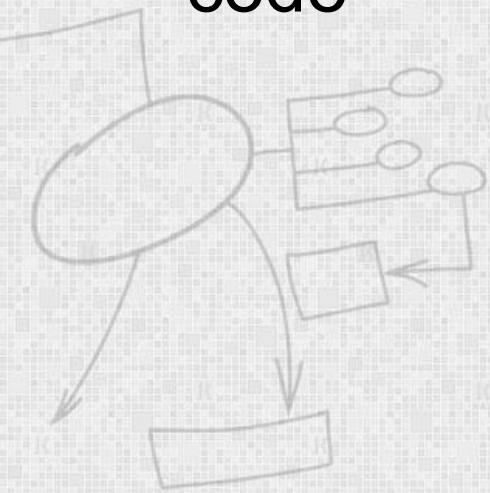


XP Practices: Continuous Integration

- Teams keep the system fully integrated at all times
- Daily, or multiple times a day builds
- Avoid 'integration hell' (Trying to integrate but unable because of integration errors)
- Avoid code freezes (Development stops for testing and fixing bugs)

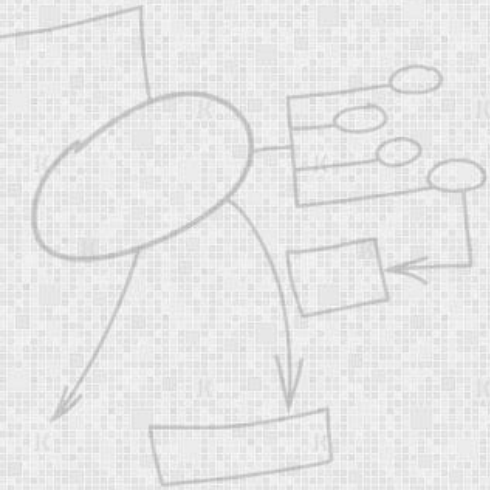
XP Practices: Collective Code Ownership

- Any pair of programmers can improve any code at any time
- All code gets the benefit of many people's attention
- Pair with expert when working on unfamiliar code



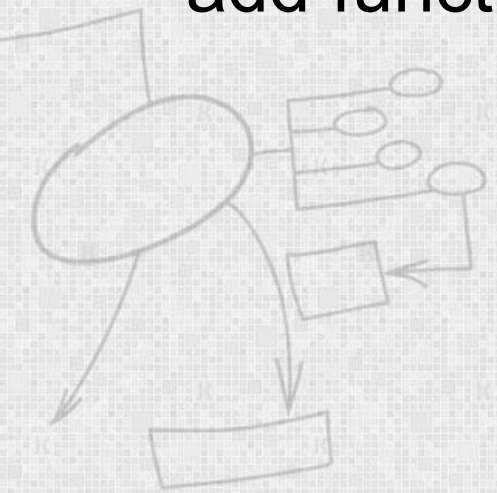
XP Practices: Coding Standard

- Use common coding standard
- All code in the system must look as though written by an individual
- Code must look familiar, to support collective code ownership



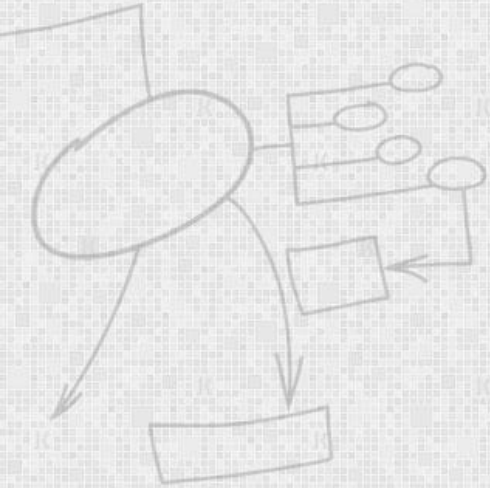
XP Practices: Metaphor

- XP Teams develop a common vision of the system
- Define common system of names
- Ensure everyone understands how the system works, where to look for functionality, or where to add functionality



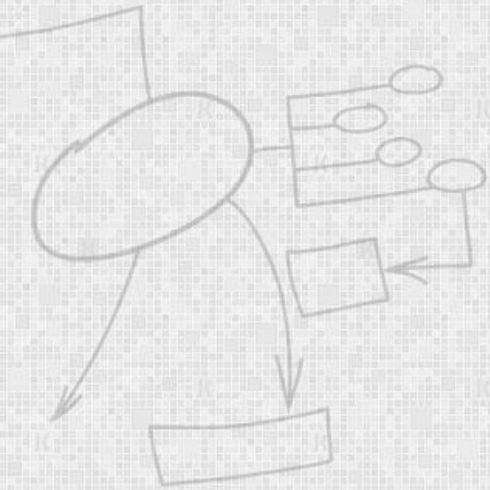
XP Practices: Sustainable Pace

- Team will produce high quality product when not overly exerted
- Avoid overtime, maintain 40 hour weeks
- Work at a pace that can be sustained indefinitely

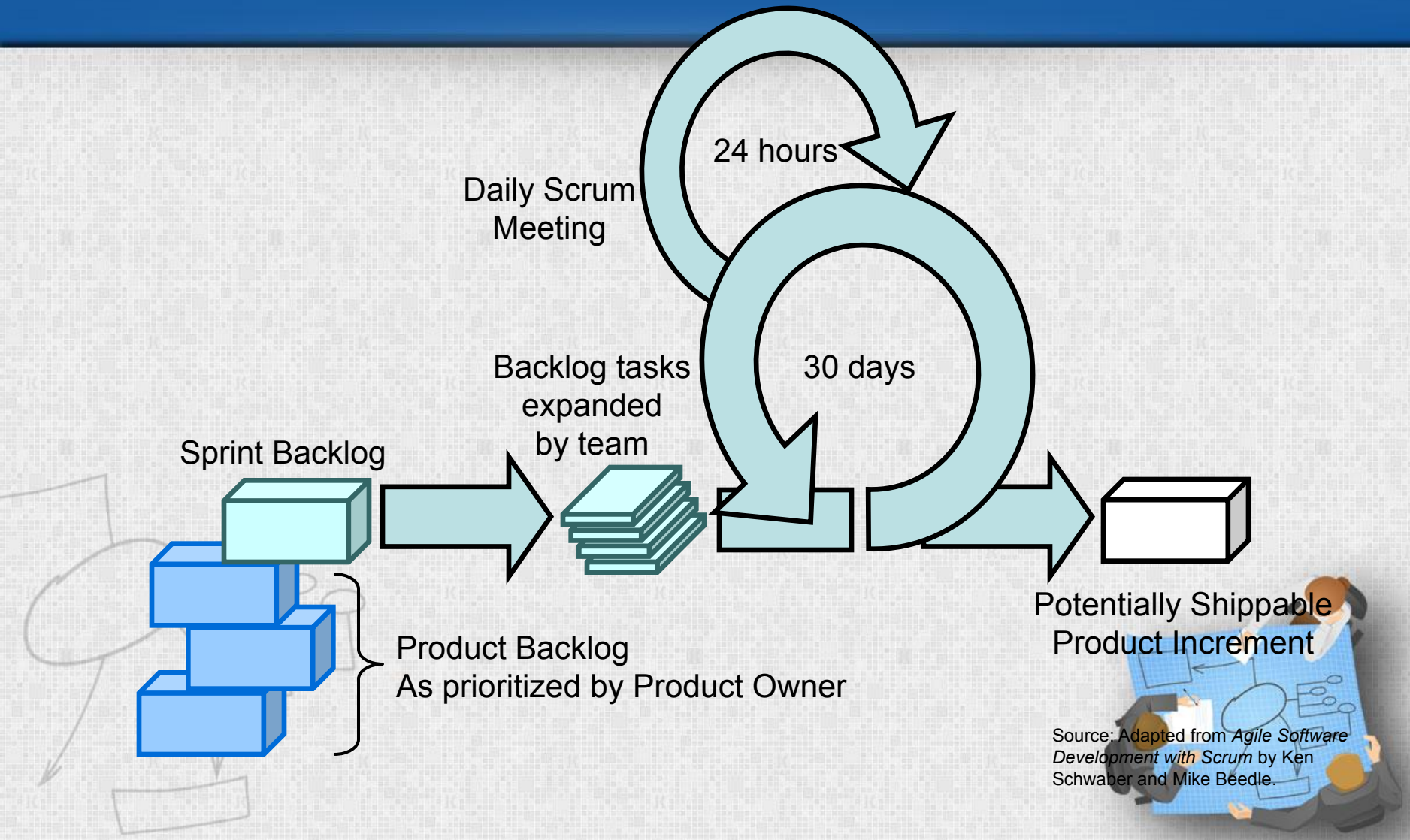


Scrum

- Originally proposed by Schwaber and Beedle
- Scrum—distinguishing features
  - Work occurs in "sprints" and is derived from a "backlog" of existing requirements
  - Meetings are very short



Scrum at a Glance



Source: Adapted from *Agile Software Development with Scrum* by Ken Schwaber and Mike Beedle.

Requirements | Design | Code | Test

Rather than doing all of one thing at a time...

...Scrum teams do a little of everything all the time

Scrum Framework

Roles

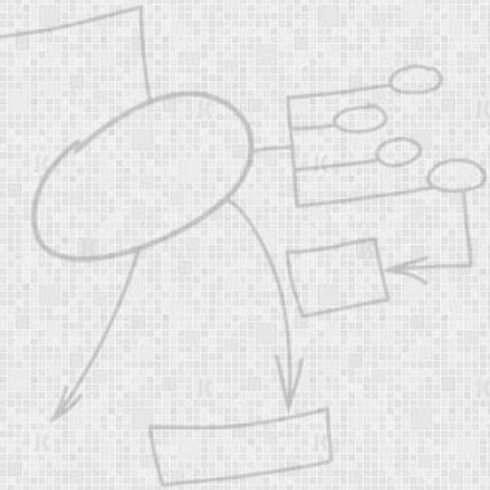
- Product owner
- Scrum Master
- Team

Ceremonies

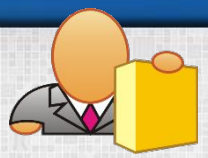
- Sprint planning
- Daily scrum meeting
- Sprint review
- Sprint retrospe

Artifacts

- Product backlog
- Sprint backlog
- Burndown charts



Scrum Roles



- Product Owner
  - Possibly a Product Manager or Project Sponsor
  - Decides features, release date, prioritization, $$$

- Scrum Master
  - Typically a Project Manager or Team Leader
  - Responsible for sanctioning Scrum values and practices
  - Remove obstacles / politics, keeps everyone productive

- Project Team
  - 5-10 members;
  - Cross-functional: QA, Programmers, UI Designers, etc.
  - Membership should change only between sprints

Sprint Planning Mtg.

Team capacity

Product backlog

Business conditions

Current product

Technology

Sprint planning meeting

Sprint prioritization

- Analyze/evaluate product backlog
- Select sprint goal

Sprint planning

- Decide how to achieve sprint goal
- Create sprint backlog (tasks) from product backlog items (user stories / features)
- Estimate sprint backlog in hours

Sprint goal

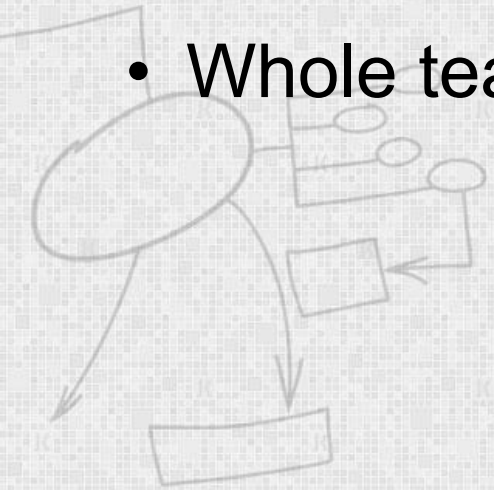
Sprint backlog

Daily Scrum Meeting

- Parameters
  - Daily, ~15 minutes, Stand-up
- Not for problem solving
  - Only team members, Scrum Master, product owner, can talk
  - Helps avoid other unnecessary meetings
- Three questions answered by each team member:
  1. What did you do yesterday?
  2. What will you do today?
  3. What obstacles are in your way?

The Sprint Review

- Team presents what it accomplished during the sprint
- Typically takes the form of a demo of new features or underlying architecture
- Informal
  - No slides
- Whole team participates

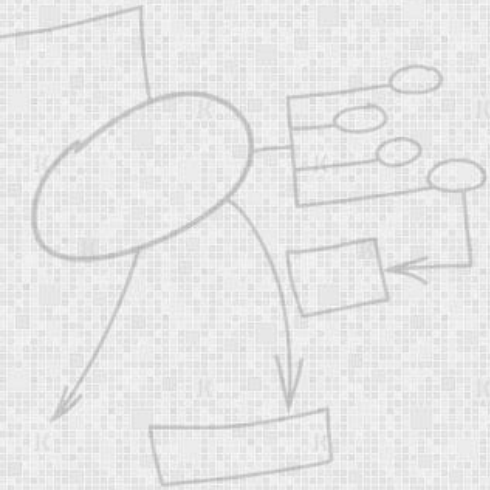


The Sprint Retrospective

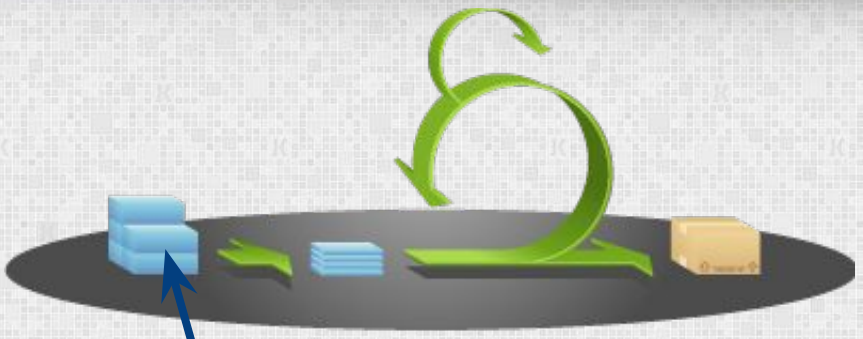
- Team discusses results of last sprint
- The retrospective includes three main questions/points for discussion:
  - What went well during the sprint cycle?
  - What went wrong during the sprint cycle?
  - What could we do differently to improve?
- Process improvements are made at the end of every sprint. This ensures that the project team is always improving the way it works.

Scrum's Artefacts

- Scrum has remarkably few artefacts
  - Product Backlog
  - Sprint Backlog
  - Burndown Charts
- Can be managed using just an Excel spreadsheet
  - More advanced / complicated tools exist:

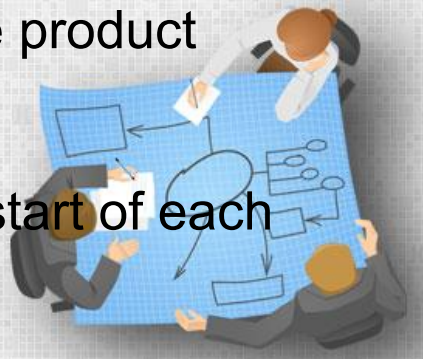


Product Backlog



This is the product backlog

- The requirements
- A list of all desired work on project
- Ideally expressed as a list of user stories along with "story points", such that each item has value to users or customers of the product
- Prioritized by the product owner
- Reprioritized at start of each sprint



User Stories

- Instead of Traditional Requirements, Agile project owners do "user stories"
  - **Who** (user role) – Is this a customer, employee, admin, etc.?
  - **What** (goal) – What functionality must be achieved/developed?
  - **Why** (reason) – Why does user want to accomplish this goal?

As a [user role], I want to [goal], so I can [reason].

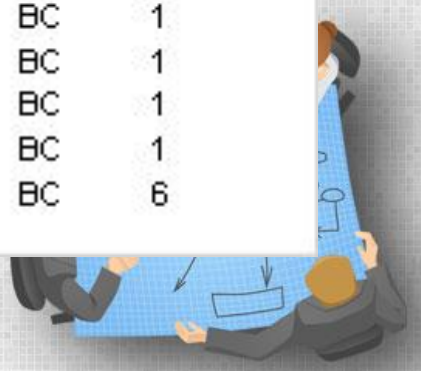
Sample Product Backlog

Backlog item	Estimate
Allow a guest to make a reservation	3 (story points)
As a guest, I want to cancel a reservation.	5
As a guest, I want to change the dates of a reservation.	3
As a hotel employee, I can run RevPAR reports (revenue-per-available-room)	8
Improve exception handling	8
…	30
…	50

Sample Product Backlog 2

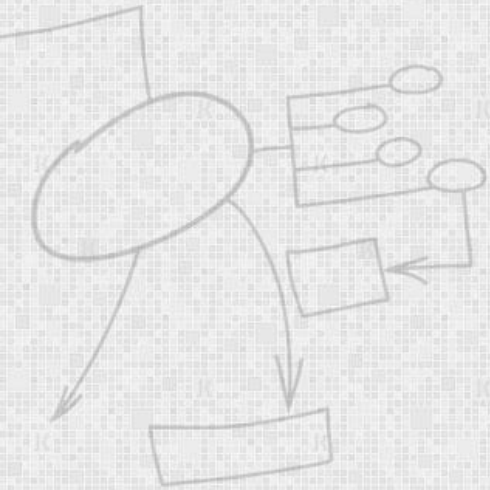
Product Backlog
Estimating System Upgrade

Sprint	ID		Backlog Item	Owner	Estimate (days)	Remaining (days)
1	1	Minor	Remove user kludge in .dpr file	BC	1	1
1	2	Minor	Remove cMap/cMenu/cMenuSize from disciplines.pas	BC	1	1
1	3	Minor	Create "Legacy" discipline node with old civils and E&I content	BC	1	1
1	4	Major	Augment each tbl operation to support network operation	BC	10	10
1	5	Major	Extend Engineering Design estimate items to include summaries	BC	2	2
1	6	Super	Supervision/Guidance	CAM	4	4
	7	Minor	Remove Custodian property from AppConfig class in globals.pas	BC	1	
	8	Minor	Remove LOC_ constants in globals.pas and main.pas	BC	1	
	9	Minor	New E&I section doesn't have lblCaption set	BC	1	
	10	Minor	Delay in main.releaseform doesn't appear to be required	BC	1	
	11	Minor	Undo modifications to Other Major Equipment in formExcel.pas	BC	1	
	12	Minor	AJACS form to be centred on the screen	BC	1	
	13	Major	Extend DUnit tests to all 40 disciplines	BC	6	



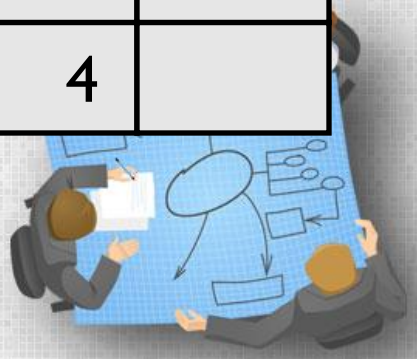
Sprint Backlog

- Individuals sign up for work of their own choosing
  - Work is never assigned
- Estimated work remaining is updated daily
- Any team member can add, delete change sprint backlog
- If work is unclear, define a sprint backlog item with a larger amount of time and break it down later



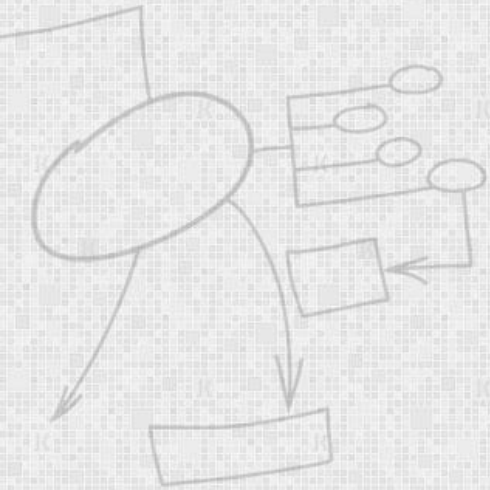
Sample Sprint backlog

Tasks	Mon	Tue	Wed	Thu	Fri
Code the user interface	8	4	8		
Code the middle tier	16	12	10	4	
Test the middle tier	8	16	16	11	8
Write online help	12				
Write the A class	8	8	8	8	8
Add error logging			8	4	

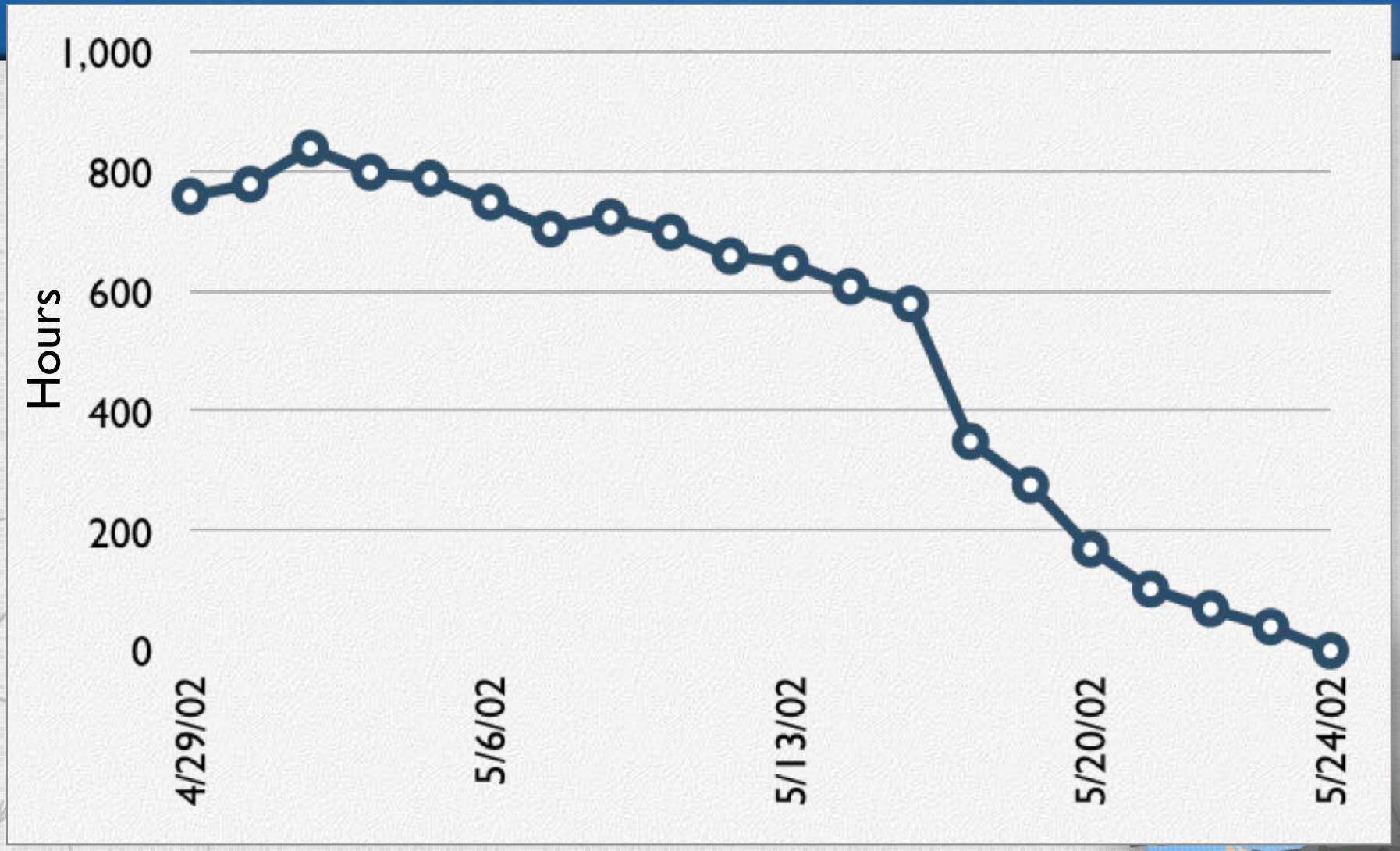


Sprint Burndown Chart

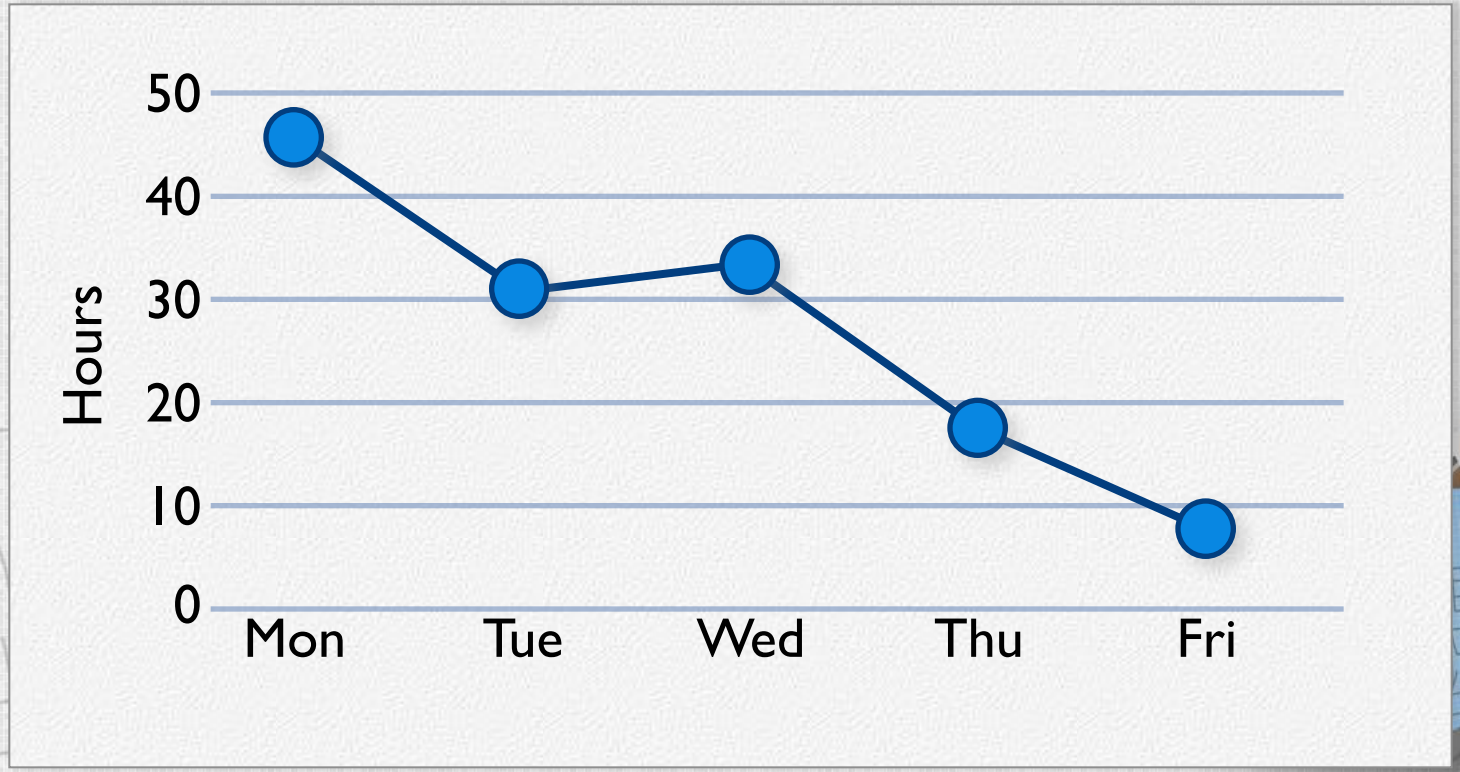
- A display of what work has been completed and what is left to complete
  - one for each developer or work item
  - updated every day
  - (make best guess about hours/points completed each day)



Sample Burndown Chart

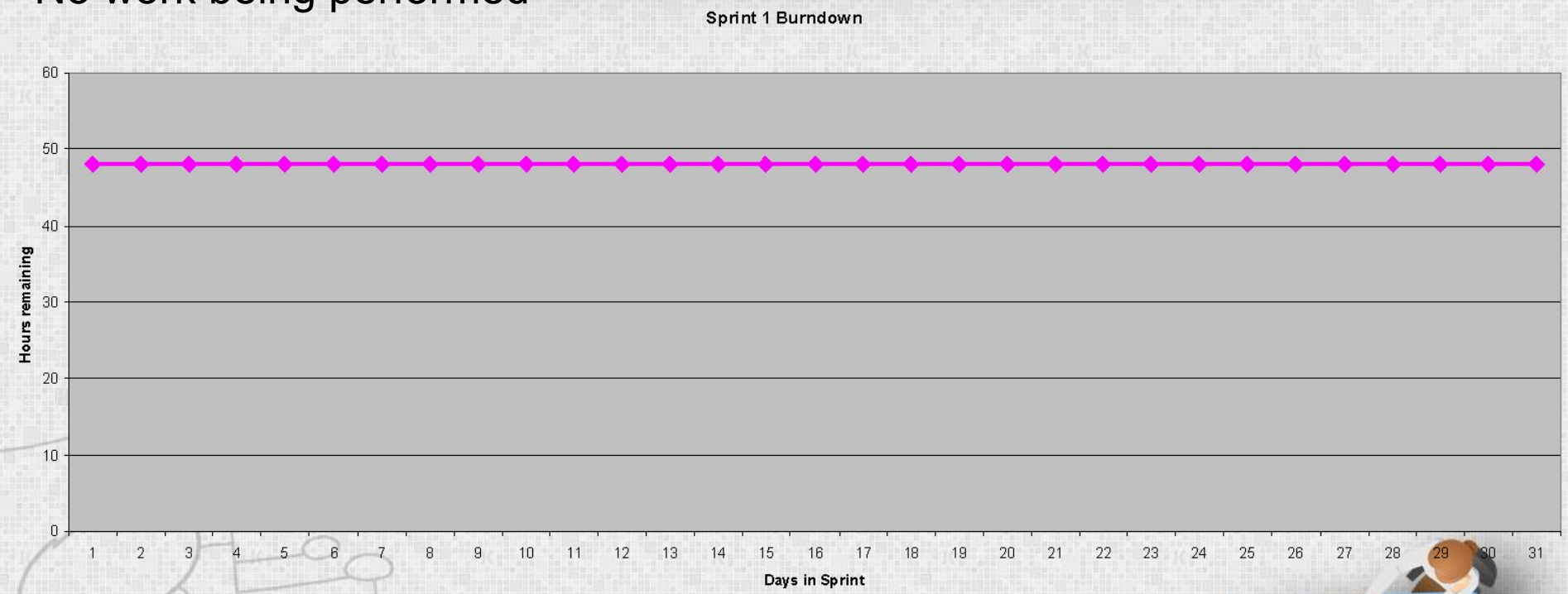


Tasks	Mon	Tue	Wed	Thu	Fri
Code the user interface	8	4	8		
Code the middle tier	16	12	10	7	
Test the middle tier	8	16	16	11	8
Write online help	12				



Burndown Example 1

No work being performed



Burndown Example 2

Work being performed, but not fast enough

Burndown Example 3

Work being performed, but too fast!